

CMP5332

Object-Oriented Programming

Unit Testing



- Compilation Errors
- Logical Errors – Runtime Errors (Bugs)
- Debugging
 - Manual Debugging
 - Automated Debugging
- JUnit Testing
 - Principles of JUnit Testing
 - Running JUnit Testing
- Examples
 - Vending Machine
 - Query String



Compilation Errors

Whenever we write some code, we need to **check that it's correct**.

Since Java is a **compiled language**, the compiler does some of these checks — for example:

- You might write the **wrong name** for a variable, method or class.
- You might use the **wrong type** of value in the wrong place.
- You might give too **few or too many arguments** to a method.
- You might forget to write a **return statement** at the end of a method.
- You might use a variable in the **wrong place**, where it isn't defined.
- You might **forget** a semicolon or some other minor syntax detail.



Compilation Errors

Every programmer makes these mistakes sometimes.

These mistakes are called “**compilation errors**” or “**compile-time errors**”.

Fortunately the **compiler detects** all of them, so they’re easy to discover and fix.

Logical or Runtime Errors

On the other hand, some mistakes are **more difficult to find** — if your code **compiles without errors**, but it does the **wrong thing** when it runs, then you won't find out the errors until you **test your code**.

These mistakes are called **“bugs”**, and almost every piece of software has some bugs.

There are lots of **different kinds of bugs**, but some common reasons for bugs are:

- Writing the **wrong logical condition** — e.g. $i < n$ instead of $i \leq n$.
- Not checking for **invalid inputs**.
- Using the **wrong standard library function** —
e.g. `Integer.getInteger` when you should have used `Integer.parseInt`.

Being a good programmer does not mean you have to write code that always works correctly on the first attempt.

Rather, a good programmer can:

- **Discover** that there is a bug,
- **Isolate** the code causing the bug,
- **Diagnose** why the isolated code causes the bug,
- **Fix** the bug, and
- **Check** that the bug has really been fixed.

This Is Called - “Debugging”

**Unit Testing helps with this
process**

The simplest way to test code is “**by hand**”; that is, by running the code to see whether it does the right thing.

Consider the following example:

```
class Vector {  
    int x;  
    int y;  
  
    Vector(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    int getX() {  
        return x;  
    }  
  
    int getY() {  
        return y;  
    }  
  
    double length() {  
        return Math.sqrt(x^2 + y^2);  
    }  
}
```

This class compiles without errors. We can test whether it behaves as we expect:

The **getX** and **getY** methods do what we expect,

but the correct length of this vector should be 5.0, which is not the result we actually got –

so we have **discovered** a bug.

```
class Vector {
    int x;
    int y;

    Vector(int x, int y) {
        this.x = x;
        this.y = y;
    }

    int getX() {
        return x;
    }

    int getY() {
        return y;
    }

    double length() {
        return Math.sqrt(x^2 + y^2);
    }
}
```

```
Vector vector = new Vector(3, 4);
vector.getX()
    3 (int)
vector.getY()
    4 (int)
vector.length()
    2.6457513110645907 (double)
```

Manual Testing

Using the numbers we've chosen, the formula $3^2 + 4^2$ should give 25, then the result of `Math.sqrt` should be 5.

The most likely cause of the bug is in the code `x^2 + y^2`, so we can try this out with the values 3 and 4:

```
3^2 + 4^2
7 (int)
```

This is not the expected result, so we have **isolated** this code as the cause of the bug.

```
class Vector {
    int x;
    int y;

    Vector(int x, int y) {
        this.x = x;
        this.y = y;
    }

    int getX() {
        return x;
    }

    int getY() {
        return y;
    }

    double length() {
        return Math.sqrt(x^2 + y^2);
    }
}
```

```
Vector vector = new Vector(3, 4);
vector.getX()
    3 (int)
vector.getY()
    4 (int)
vector.length()
    2.6457513110645907 (double)
```

Manual Testing

The expected behaviour of this code is that 3^2 should be 9, then 4^2 should be 16, then $9 + 16$ should be 25.

However, using the separate tests we can see that the $^$ operator does not do what we expect:

```
3^2
1 (int)
4^2
6 (int)
```

Now we have **diagnosed** the bug — it is caused by an incorrect use of the $^$ operator.

```
class Vector {
    int x;
    int y;

    Vector(int x, int y) {
        this.x = x;
        this.y = y;
    }

    int getX() {
        return x;
    }

    int getY() {
        return y;
    }

    double length() {
        return Math.sqrt(x^2 + y^2);
    }
}
```

```
Vector vector = new Vector(3, 4);
vector.getX()
3 (int)
vector.getY()
4 (int)
vector.length()
2.6457513110645907 (double)
```

Manual Testing

The code must be changed so that it correctly squares x and y . The simplest solution is to write $x*x$ instead of x^2 :

```
double length() {  
    return Math.sqrt(x*x + y*y);  
}
```

After compiling the class again, we now believe we have **fixed** the bug.

```
class Vector {  
    int x;  
    int y;  
  
    Vector(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    int getX() {  
        return x;  
    }  
  
    int getY() {  
        return y;  
    }  
  
    double length() {  
        return Math.sqrt(x^2 + y^2);  
    }  
}
```

```
Vector vector = new Vector(3, 4);  
vector.getX()  
    3 (int)  
vector.getY()  
    4 (int)  
vector.length()  
    2.6457513110645907 (double)
```

To be sure, we should **check** that the method now works correctly. We can test the code again for this::

```
Vector vector = new Vector(3, 4);  
vector.length()  
5.0 (double)
```

Now we get the expected output, so we have some confidence that this bug is fixed.

However, there may still be other bugs; more testing will be required.

```
class Vector {  
    int x;  
    int y;  
  
    Vector(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    int getX() {  
        return x;  
    }  
  
    int getY() {  
        return y;  
    }  
  
    double length() {  
        return Math.sqrt(x^2 + y^2);  
    }  
}
```

```
Vector vector = new Vector(3, 4);  
vector.getX()  
3 (int)  
vector.getY()  
4 (int)  
vector.length()  
2.6457513110645907 (double)
```

Fixing a bug — and checking that it's fixed — is not the end of the story.

In real software, bugs often appear in code because of **later changes** or even **changes in other classes**. These are called regression bugs.

We don't want to have to go through the same **manual testing process** every time the `Vector` class changes; manual testing is **fairly easy**, **but** it's **slow**, and we **might make mistakes**.

Anything that can be **automated** and is **repetitive** should be done by the computer instead.

**Let's Write a Program Which Does Our
Testing Automatically**

Automated Testing

```
1: class VectorTest {
2:     static void main() {
3:         Vector vector = new Vector(3, 4);
4:
5:         int x = vector.getX();
6:         System.out.print("Testing getX() method: ");
7:         if(x == 3) {
8:             System.out.println("passed");
9:         } else {
10:            System.out.println("failed, expected 3, was " + x);
11:        }
12:
13:        int y = vector.getY();
14:        System.out.print("Testing getY() method: ");
15:        if(y == 4) {
16:            System.out.println("passed");
17:        } else {
18:            System.out.println("failed, expected 4, was " + y);
19:        }
20:
21:        double length = vector.length();
22:        System.out.print("Testing length() method: ");
23:        if(length == 5.0) {
24:            System.out.println("passed");
25:        } else {
26:            System.out.println("failed, expected 5.0, was " + length);
27:        }
28:    }
29: }
```

```
class Vector {
    int x;
    int y;

    Vector(int x, int y) {
        this.x = x;
        this.y = y;
    }

    int getX() {
        return x;
    }

    int getY() {
        return y;
    }

    double length() {
        return Math.sqrt(x2 + y2);
    }
}
```



The `VectorTest` program we wrote is helpful because it can do our tests automatically.

However, it took a lot of **duplicate code** to write.

Since writing automated tests is so important, we can save a lot of work by using a **testing framework** which handles the boring parts.

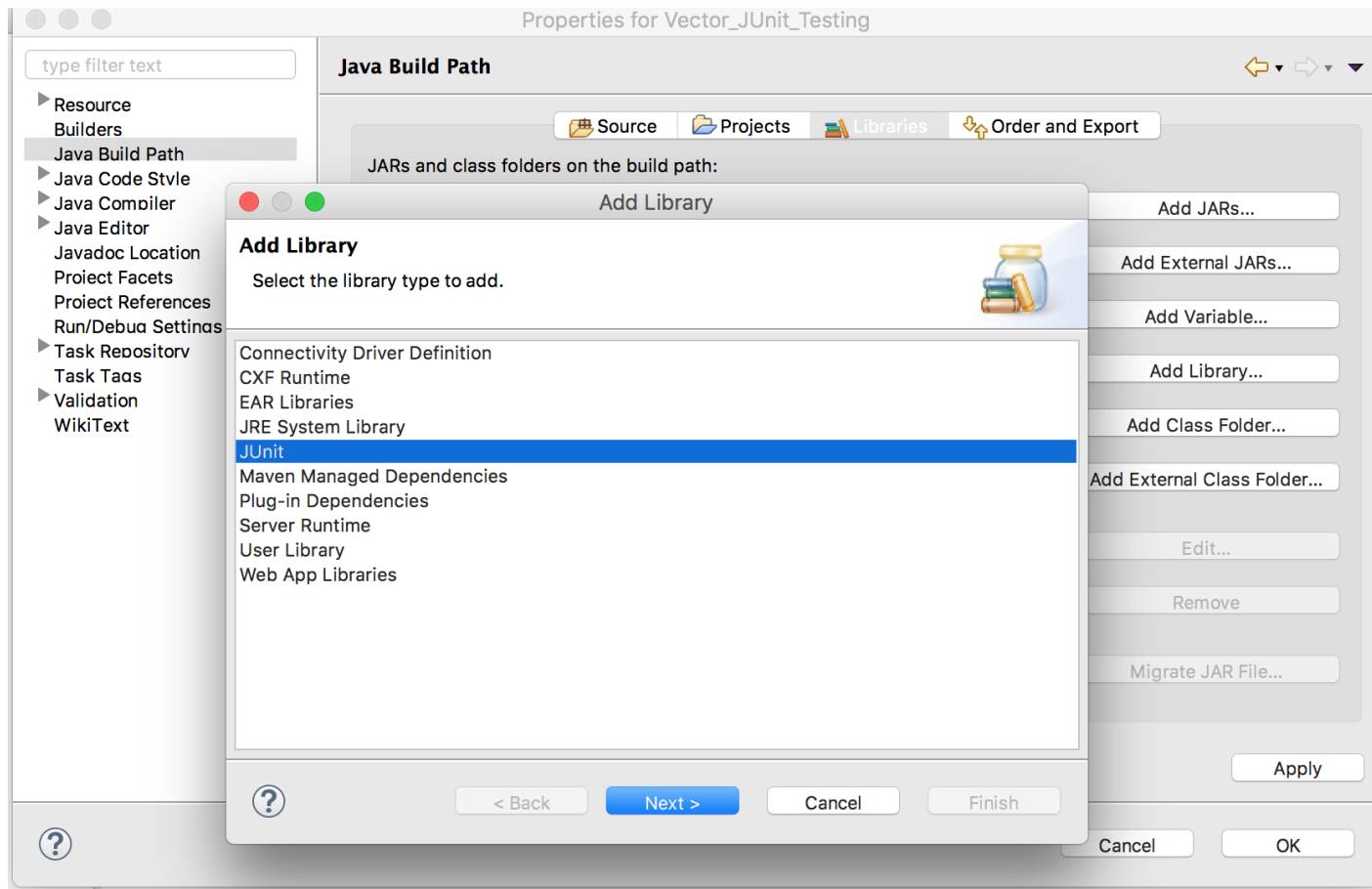
We'll Use

JUnit Testing



The Junit Testing Framework

How to Download and Install JUnit in Eclipse



Here's what a JUnit test class looks like:

```
1: import org.junit.Test;
2: import static org.junit.Assert.assertEquals;
3:
4: public class VectorTest {
5:     @Test
6:     public void testGetX() {
7:         Vector vector = new Vector(3, 4);
8:         assertEquals(3, vector.getX());
9:     }
10:
11:     @Test
12:     public void testGetY() {
13:         Vector vector = new Vector(3, 4);
14:         assertEquals(4, vector.getY());
15:     }
16:
17:     @Test
18:     public void testLength() {
19:         Vector vector = new Vector(3, 4);
20:         // error tolerance of 0.0 - result must be exact
21:         assertEquals(5.0, vector.length(), 0.0);
22:     }
23: }
```

There are a few new things to explain in this code:

- The import declarations at the top allow us to use **Test** and (**assertEquals**) from the JUnit “package”.
- The **assertEquals** import is static since it's a **static** method.
- The keyword **public** allows JUnit to access our code. Since JUnit itself is in a separate “package”, it can only see our class and methods if we make them “public”.
- JUnit uses the **@Test** annotation to know which methods are tests.

Every test should make at least one **assertion** about what the result of the test should be.

In the `VectorTest` class, the `assertEquals` method is used three times:

- To assert that **3** is returned by the `getX` method.
- To assert that **4** is returned by the `getY` method.
- To assert that **5.0** is returned by the `length` method.

The `assertEquals` method checks whether the **expected value** (first argument) equals the **actual value** (second argument).

If they are not equal, the test will fail.

```
1: import org.junit.Test;
2: import static org.junit.Assert.assertEquals;
3:
4: public class VectorTest {
5:     @Test
6:     public void testGetX() {
7:         Vector vector = new Vector(3, 4);
8:         assertEquals(3, vector.getX());
9:     }
10:
11:     @Test
12:     public void testGetY() {
13:         Vector vector = new Vector(3, 4);
14:         assertEquals(4, vector.getY());
15:     }
16:
17:     @Test
18:     public void testLength() {
19:         Vector vector = new Vector(3, 4);
20:         // error tolerance of 0.0 - result must be exact
21:         assertEquals(5.0, vector.length(), 0.0);
22:     }
23: }
```

When comparing double values, a third argument is needed for the **error tolerance**.

In this example, the error tolerance is **0.0** because we want the result to be exactly 5.0.

However, since double has a limited precision, calculations often don't give exact results, so usually we should allow a small error.

For example, an error tolerance of **0.5e-10** means the result must be accurate to 10 decimal places:

```
@Test
public void testLength() {
    Vector vector = new Vector(3, 4);
    // error tolerance of 0.0 - result must be exact
    assertEquals(5.0, vector.length(), 0.0);
}
```

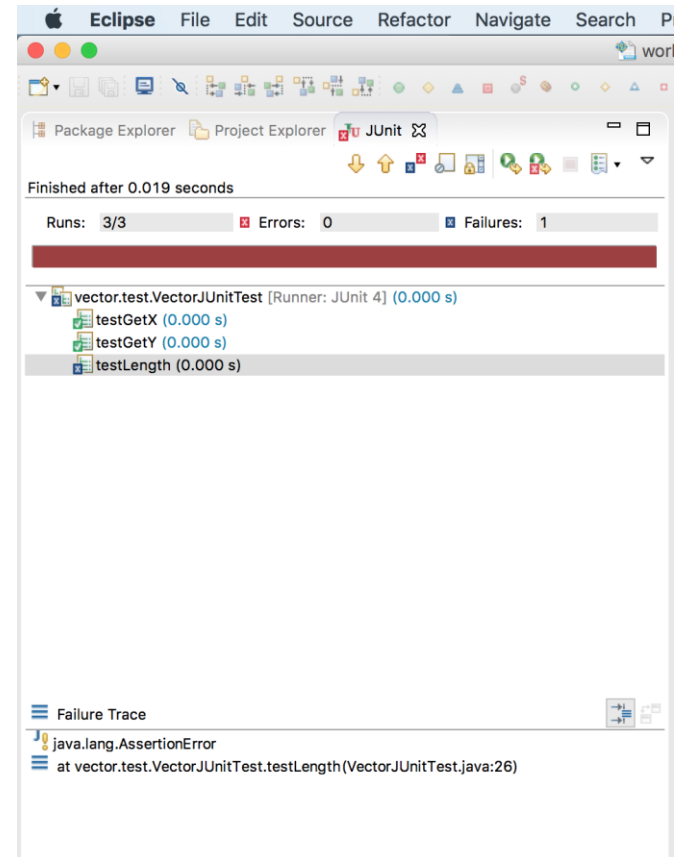
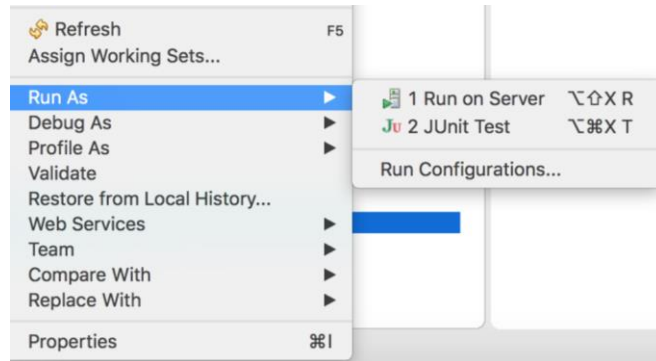
Some other useful JUnit assertion methods are:

- `assertTrue` tests that some condition is true.
- `assertFalse` tests that some condition is false.
- `assertNull` tests that some reference is null.
- `assertNotNull` tests that some reference is not null.
- `assertNotEquals` tests that two values are different.
- `fail` causes the test to fail unconditionally. This should be used to test that a particular line of code will not be reached.

```
@Test
public void testLength() {
    Vector vector = new Vector(3, 4);
    assertTrue(5.0 == vector.length());
}
```

Running JUnit tests

To run the tests from Eclipse, simply right-click on the `VectorTest` class and select “Run As – JUnit Test”



From here we can see that the first two tests passed, but the test of the `length` method failed (just as it did when we tested manually).

Principles of Unit Testing

A “**unit**” is the smallest testable amount of code, so this testing strategy is called “unit testing”.

A “**unit test suite**” is a collection of unit tests, grouped into one or more test classes.

A “**test case**” means

- the **input data** used in a test,
- the **action** to be performed with that data, and
- the **expected result**.

For example, the test cases we used to test the Vector class are:

Input	Action	Expected result
<code>new Vector(3, 4)</code>	call <code>getX</code> method	It returns <code>3</code> .
<code>new Vector(3, 4)</code>	call <code>getY</code> method	It returns <code>4</code> .
<code>new Vector(3, 4)</code>	call <code>length</code> method	It returns <code>5.0</code> .

```

class VendingMachine {
    static final int CANDY_PRICE = 3;
    static final int CANDY_CAPACITY = 20;

    int candyBars;
    int balance;
    int revenue;

    VendingMachine() {
        candyBars = CANDY_CAPACITY;
        balance = 0;
        revenue = 0;
    }

    int getBalance() {
        return balance;
    }

    int getRevenue() {
        return revenue;
    }

    void insertCoin() {
        balance++;
    }

    int refund() {
        int amount = balance;
        balance = 0;
        return amount;
    }
}

```

```

boolean vendCandyBar() {
    if(candyBars >= 1 && balance >= CANDY_PRICE) {
        candyBars--;
        balance -= CANDY_PRICE;
        revenue += CANDY_PRICE;
        return true;
    } else {
        return false;
    }
}

void restock() {
    candyBars = CANDY_CAPACITY;
}
}

```

- Create a new VendingMachine Object
- Insert 5 coins
- Check the balance
- By a candy bar
- Check balance and revenue

Query String



The image shows a web browser window with a login form. The form is titled "Log In" and contains two input fields: "Username" and "Password". The "Username" field contains the text "AR" and the "Password" field contains the text "mypass!". Below the input fields is a button labeled "Log In".

<http://www.test.org/login?username=AR&password=mypass%21>

<http://www.test.org/login?username=AR&password=mypass%21>

```
/*
 * Represents a query string, such as "foo=bar&baz=1234".
 * See https://en.wikipedia.org/wiki/Query_string
 */
class QueryString {
    String data;

    QueryString(String data) {
        this.data = data;
    }

    /*
     * Returns the value associated with a parameter in the query string,
     * or null if the parameter does not exist.
     */
    String getParameter(String name) {
        int offset = getParameterOffset(name);
        if(offset == -1) {
            return null;
        }

        // get the position after the next = symbol
        int begin = data.indexOf('=', offset) + 1;

        // get the position of the next & symbol
        int end = data.indexOf('&', begin);
        if(end == -1) {
            // if there is no & symbol, this is the last parameter,
            // so read to the end of the string
            end = data.length();
        }

        String value = data.substring(begin, end);
        return decode(value);
    }

    /*
     * Determines whether a parameter exists in the query string.
     */
    boolean hasParameter(String name) {
        return getParameterOffset(name) >= 0;
    }
}
```

```
/*
 * Returns the character offset of the start of a parameter's name.
 * If the parameter does not exist, -1 is returned.
 */
int getParameterOffset(String name) {
    if(data.startsWith(name + '=')) {
        // parameter appears at the start of the query string
        return 0;
    }

    int offset = data.indexOf('&' + name + '=');
    if(offset != -1) {
        // the parameter exists, so skip the & symbol
        offset++;
    }
    return offset;
}

/*
 * Decodes a URL-encoded string.
 * See https://en.wikipedia.org/wiki/Percent-encoding
 */
static String decode(String s) {
    String result = "";
    int i = 0;
    while(i < s.length()) {
        char c = s.charAt(i);
        if(c == '%') {
            // decode two hexadecimal digits after the % symbol
            String hex = s.substring(i+1, i+3);
            int charCode = Integer.parseInt(hex, 16);
            c = (char) charCode;
            // skip two more characters
            i += 2;
        } else if(c == '+') {
            // + symbol is decoded to a space
            c = ' ';
        }
        result += c;
        i++;
    }
    return result;
}
```

- Create a new QueryString Object
- Check if the query string has a parameter password
- Determine the index of password
- Get the parameter for password